

Linux Server Monitoring Project

How to Use the Project — From Setup to Running and Checking Results

This guide is specifically for the uploaded Linux Server Monitoring Python program. It explains exactly what you need, how to prepare PostgreSQL, how to configure the program, how to run it, what output to expect, how to verify the database and Excel results, how to test alerts, and how to stop the program.

1. What this project does

The program continuously monitors the computer/server where it is running. It collects CPU usage, memory usage and disk usage using psutil. It checks those values against configured thresholds, stores each measurement in PostgreSQL, writes the same measurement to a daily Excel file, and can send a webhook alert when a threshold is exceeded. The default monitoring interval is 60 seconds.

The main flow is:

```
graph TD
    Start --> Connect[Connect to PostgreSQL]
    Connect --> Create[Create server_metrics table if needed]
    Create --> Collect[Collect CPU / Memory / Disk]
    Collect --> Check[Check alert thresholds]
    Check --> SaveDB[Save to PostgreSQL]
    SaveDB --> SaveExcel[Save to Excel]
    SaveExcel --> Display[Display status]
    Display --> Wait[Wait 60 seconds]
    Wait --> Repeat[Repeat]
```

2. What you need before starting

Install Python and PostgreSQL on your computer. You also need the Python packages used by the uploaded program: psutil, psycopg2-binary, openpyxl and requests.

Check Python:

```
python --version
```

Check PostgreSQL:

```
psql --version
```

3. Create the project folder

```
mkdir linux-server-monitoring
cd linux-server-monitoring
```

Put the uploaded Python file into this folder and rename it to server_monitor.py.

4. Create a virtual environment

```
python -m venv venv
```

On Windows, activate it with:

```
venv\Scripts\activate
```

After activation, the terminal normally shows (venv) before the command prompt.

5. Install the required packages

```
pip install psutil psycpg2-binary openpyxl requests
```

These correspond to the libraries imported by the uploaded program: psutil for system metrics, psycpg2 for PostgreSQL, openpyxl for Excel, and requests for optional webhook requests.

6. Create the PostgreSQL database

Open pgAdmin or the PostgreSQL command line and create a database named linux_monitor.

```
CREATE DATABASE linux_monitor;
```

The program will create the server_metrics table itself when it starts. You do not need to manually create that table for the normal application flow.

7. Configure the database connection

The uploaded source currently contains database connection settings. Before putting the project on GitHub, do not keep a real password inside the Python source. Use environment variables instead.

Use this configuration in server_monitor.py:

```
import os

HOST = os.getenv("DB_HOST", "localhost")
DATABASE = os.getenv("DB_NAME", "linux_monitor")
USER = os.getenv("DB_USER", "postgres")
PASSWORD = os.getenv("DB_PASSWORD")
PORT = os.getenv("DB_PORT", "5432")
```

For local testing, your environment values should correspond to your PostgreSQL installation:

```
DB_HOST=localhost
DB_NAME=linux_monitor
DB_USER=postgres
DB_PASSWORD=YOUR_POSTGRES_PASSWORD
DB_PORT=5432
```

8. Understand the table created by the program

At startup, create_table() creates server_metrics if it does not exist.

```
CREATE TABLE IF NOT EXISTS server_metrics (
    id SERIAL PRIMARY KEY,
    cpu_usage DECIMAL(5,2) NOT NULL,
    memory_usage DECIMAL(5,2) NOT NULL,
    disk_usage DECIMAL(5,2) NOT NULL,
    recorded_at TIMESTAMP NOT NULL
);
```

Each monitoring cycle inserts one row containing CPU, memory, disk and timestamp.

9. Run the project

Make sure the virtual environment is active, PostgreSQL is running, and the linux_monitor database exists.

```
python server_monitor.py
```

The program should first display a startup section similar to:

```
=====
                        SERVER MONITORING SYSTEM
=====
Operating System : ...
Disk Monitoring  : ...
Database         : linux_monitor
Excel Mode       : Daily File Rotation
Interval        : 60 seconds
=====

PostgreSQL table ready.

Monitoring started... Press Ctrl + C to stop.
```

10. What happens every 60 seconds

The main loop performs these steps in order:

1. Get the current timestamp
2. Collect CPU usage
3. Collect memory usage
4. Collect disk usage
5. Check CPU/memory/disk thresholds
6. Print alerts if necessary
7. Save metrics to PostgreSQL
8. Save metrics to the daily Excel file
9. Display DB and Excel save status
10. Wait for the configured interval
11. Repeat

11. Understand the terminal output

A normal status line looks approximately like:

```
2026-08-15 10:00:00 | CPU: 24.30% | Memory: 58.20% | Disk: 72.10% | DB: OK | Excel: OK
```

CPU, Memory and Disk are the collected utilization percentages. DB: OK means the PostgreSQL insert succeeded. Excel: OK means the Excel file was successfully written.

12. Check the PostgreSQL result

Open pgAdmin and select the linux_monitor database, then query the table:

```
SELECT * FROM server_metrics;
```

You should see rows such as:

id	cpu_usage	memory_usage	disk_usage	recorded_at
1	24.30	58.20	72.10	2026-08-15 ...
2	18.50	57.90	72.10	2026-08-15 ...

13. Check the Excel result

The program creates a daily Excel file named Server_Monitoring_YYYY-MM-DD.xlsx. For example: Server_Monitoring_2026-08-15.xlsx.

The worksheet is named **Server Monitoring** and contains:

```
ID
Date
Time
CPU %
Memory %
Disk %
Timestamp
```

The program creates the file if it does not exist and appends a new monitoring record during each cycle.

14. Test the alert system

The uploaded program has default thresholds of CPU 85%, memory 90% and disk 90%. For a safe local test, temporarily lower one threshold, run the program, verify the alert, and then restore it.

```
# Temporary test
CPU_THRESHOLD = 10.0

# Restore after testing
CPU_THRESHOLD = 85.0
```

An alert can look like:

```
[ALERT - 2026-08-15 10:05:00]
- HIGH CPU USAGE: 25.40% (Threshold: 10.0%)
```

After the test, restore the original threshold. The project also contains an alert cooldown so that webhook notifications are not sent continuously.

15. Test the webhook feature

The uploaded code supports an optional webhook URL. If WEBHOOK_URL is empty, no webhook is sent. If you configure a webhook endpoint, the program sends a JSON payload when an alert occurs and the cooldown permits it.

```
WEBHOOK_URL =
```

Do not publish private webhook URLs. Keep them in environment variables if you use a real webhook.

16. Stop the project

Press:

```
Ctrl + C
```

The program catches KeyboardInterrupt, prints that monitoring stopped, shows the number of records collected during the session, and exits.

17. Common problems

Problem: PostgreSQL connection error

Check that PostgreSQL is running, the database linux_monitor exists, the username/password are correct, and port 5432 is correct.

Problem: ModuleNotFoundError

```
pip install psutil psycpg2-binary openpyxl requests
```

Problem: Excel says it is open

Close the generated Excel file and let the program write to it again.

Problem: No alert appears

For testing, temporarily lower the relevant threshold and make sure the program is running.

18. After the project works locally

Your development folder can be organized as:

```
linux-server-monitoring/  
■■■ server_monitor.py  
■■■ requirements.txt  
■■■ README.md  
■■■ .gitignore  
■■■ .env.example  
■■■ sql/  
■   ■■■ schema.sql  
■■■ screenshots/  
    ■■■ terminal.png  
    ■■■ postgresql.png  
    ■■■ excel.png
```

19. Create requirements.txt

```
psutil  
psycpg2-binary  
openpyxl  
requests
```

20. Create .gitignore

```
__pycache__/  
*.pyc  
venv/  
.venv/  
.env  
*.xlsx  
*.xls  
*.log  
.vscode/  
.idea/
```

This is important because your password, virtual environment, generated Excel reports and logs should not be committed.

21. Create .env.example

```
DB_HOST=localhost  
DB_NAME=linux_monitor  
DB_USER=postgres  
DB_PASSWORD=your_password_here  
DB_PORT=5432  
WEBHOOK_URL=
```

Upload .env.example, but never upload the real .env file.

22. Upload the project to GitHub

1. Create a GitHub repository named linux-server-monitoring.
2. Open a terminal in your project folder.

```
git init  
git add .  
git commit -m "Initial Linux server monitoring project"  
git branch -M main  
git remote add origin https://github.com/YOUR_USERNAME/linux-server-monitoring.git  
git push -u origin main
```

Replace YOUR_USERNAME with your GitHub username. Complete GitHub authentication if requested.

23. What your GitHub README should tell the user

```
# Linux Server Monitoring System

## Overview
Monitors CPU, memory and disk utilization.

## Features
- CPU monitoring
- Memory monitoring
- Disk monitoring
- PostgreSQL storage
- Daily Excel reports
- Threshold alerts
- Optional webhook alerts

## Installation
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt

## Database Setup
CREATE DATABASE linux_monitor;

## Configuration
Set DB_HOST, DB_NAME, DB_USER, DB_PASSWORD and DB_PORT.

## Run
python server_monitor.py

## Results
PostgreSQL stores metrics and Excel stores daily reports.
```

24. Final test before publishing

```
[ ] PostgreSQL is working
[ ] linux_monitor database exists
[ ] Python environment works
[ ] All packages install
[ ] server_monitor.py runs
[ ] PostgreSQL rows are created
[ ] Excel file is created
[ ] Alerts can be tested
[ ] Real password is not in source code
[ ] .env is ignored
[ ] venv is ignored
[ ] Generated Excel files are ignored
[ ] README.md exists
[ ] requirements.txt exists
[ ] GitHub repository opens correctly
```

25. Important note about this project

Based on the uploaded implementation, this is a local system-resource monitoring application: it monitors the machine where the script runs. Do not describe it as a multi-server monitoring platform unless you extend it to monitor remote servers.